郑州轻工业大学

本科毕业设计（论文）

题　　目　校园物资智能管理系统设计与实现

学生姓名

专业班级

学　　号

学　　院

指导教师(职称)

完成时间　　20　年　月　　日

郑州轻工业大学

本科毕业设计（论文）任务书

题目　　校园物资智能管理系统设计与实现

专业　　学号　　姓名

主要内容、基本要求、主要参考资料等：

一、 主要内容

本课题围绕校园物资管理信息化需求，设计并实现“校园物资智能管理系统”。系统采用前后端分离架构，后端基于 Spring Boot + MyBatis-Plus + MySQL，前端基于 Vue3 + Vite，主要完成以下内容：

1. 完成系统需求分析、总体架构设计与数据库设计；
2. 实现基础数据管理（用户、角色、部门、校区、仓库、库位、物资分类与档案）；
3. 实现核心业务功能（入库、出库、调拨、申领审批、库存批次与效期管理）；
4. 实现预警与分析功能（低库存、临期预警、统计报表与可视化）；
5. 实现系统安全与审计（基于 Spring Security + JWT 的认证授权、登录日志与操作日志）；
6. 完成系统测试、性能优化与毕业论文撰写。

二、基本要求

1. 系统功能完整，业务流程清晰，满足校园物资管理实际应用场景；
2. 数据库设计规范，关键字段与索引合理，具备基本并发与查询性能；
3. 前后端交互稳定，主要页面与接口运行正常，具备较好可用性；
4. 代码结构清晰、命名规范、注释完整，具备可维护性与可扩展性；
5. 毕业论文内容完整、逻辑严谨、格式规范，符合学校本科论文要求。

三、思政要求

1. 坚持服务校园治理与师生需求，体现信息化建设的社会责任；
2. 强化数据安全与权限边界意识，落实最小权限和审计追踪机制；
3. 在系统设计中体现协同管理、规范管理和应急保障意识；
4. 培养严谨求实、诚信科研、规范开发的职业素养。

四、主要参考资料

[1] 王伟, 刘明. 基于 Spring Boot 的后端架构设计与性能优化研究[J]. 计算机系统应用, 2021, 30(5): 102-109.
[2] 陈志敏, 何建国. 大数据背景下智慧校园建设与管理模式创新探讨[J]. 现代教育管理, 2020, 12(8): 45-51.
[3] 张强, 李红梅. 高校突发公共事件应急物资储备体系构建研究[J]. 中国安全生产科学技术, 2022, 18(3): 67-73.
[4] 李东, 赵莉. Vue3+Vite 开发实战: 组合式 API 与工程化实践[M]. 北京: 人民邮电出版社, 2023: 155-180.
[5] 刘强, 周波. MyBatis-Plus 企业级高可用开发指南[M]. 电子工业出版社, 2021: 89-105

完 成 期 限：

指导教师签名：

专业负责人签名：

年　　月　　日

校园物资智能管理系统设计与实现

摘　　要

面对高校物资台账分散、审批流转偏长、库存批次难以追踪和预警处置滞后的问题，本文围绕当前项目仓库完成了校园物资智能管理系统的设计与实现。系统并不追求脱离项目的概念包装，而是以已有代码、SQL、配置和文档为边界：前端采用 Vue 3、Pinia、Element Plus 和 ECharts 组织页面与图表，后端采用 Spring Boot 3、Spring Security、JWT 和 MyBatis-Plus 完成认证授权、业务编排与数据访问，数据层使用 MySQL 8 存储业务数据。围绕这一实现基础，论文重点说明了基础数据管理、库存与批次维护、申领审批、出库执行、仓间调拨、预警扫描、补货建议、通知消息和日志审计等已落地模块，并对统一响应结构、角色权限边界、刷新令牌续期和按到期日优先出库等关键机制进行了归纳。测试部分只采用当前仓库可以复核的事实，记录了 2026 年 4 月 14 日后端 42 项自动化测试通过、前端生产构建成功的结果。这些证据表明，当前项目已经具备覆盖校园物资核心流程的实现基础，论文中的结论也据此限定在真实可验证的范围内。

关键词：校园物资管理；Spring Boot；Vue 3；库存预警；申领审批

Design and Implementation of Campus Material Intelligent Management System

ABSTRACT

This thesis addresses the problems of scattered ledgers, lengthy approval chains, weak batch traceability, and delayed warning handling in campus material management. The discussion stays within the boundaries of the current repository rather than relying on generic system narratives. On the client side, Vue 3, Pinia, Element Plus, and ECharts are used to organize business pages and analytical views. On the server side, Spring Boot 3, Spring Security, JWT, and MyBatis-Plus are used to implement authentication, authorization, workflow orchestration, and data access, with MySQL 8 as the persistence layer. Based on these verified assets, the thesis focuses on master data maintenance, inventory and batch tracking, application approval, outbound execution, warehouse transfer, warning generation, replenishment suggestion, notification delivery, and audit logging. It also summarizes the actual mechanisms behind the unified response contract, role-based access control, refresh-token renewal, and expiry-oriented outbound deduction. The testing chapter is grounded only in executable evidence from the repository, including the backend test suite and the frontend production build. On April 14, 2026, 42 backend tests passed and the frontend build completed successfully. These results indicate that the current project already provides an implementable basis for the core workflow of campus material management.

Keywords: campus material management; Spring Boot; Vue 3; inventory warning; application approval

摘　　要 1
ABSTRACT 2
1　绪论 6
1.1 研究背景 6
1.2 研究意义 6
1.3 国内外研究现状 7
1.3.1 国内研究现状 7
1.3.2 国外相关实践启示 7
1.4 研究内容与论文结构 7
2　系统需求分析 9
2.1 业务场景与角色划分 9
2.2 功能需求分析 9
2.2.1 基础数据管理需求 9
2.2.2 库存与批次管理需求 9
2.2.3 申领审批与出库需求 10
2.2.4 调拨协同需求 10
2.2.5 预警、统计与支撑功能需求 10
2.3 非功能需求分析 11
2.4 业务流程与用例分析 11
2.4.1 申领审批用例分析 11
2.4.2 调拨执行用例分析 12
2.4.3 预警处置与统计分析用例分析 12
3　系统总体设计 13
3.1 技术选型与总体架构 13

3.1.1 技术选型依据 13
3.1.2 系统总体架构 13
3.2 功能模块设计 14
3.3 数据库设计 15
3.3.1 数据库设计原则 15
3.3.2 实体关系设计 16
3.3.3 关键数据表设计 18
3.4 接口与安全设计 22
3.4.1 接口设计 22
3.4.2 认证与授权设计 22
3.4.3 安全与审计设计 23
4 系统详细设计与实现 24
4.1 开发环境与运行环境 24
4.2 认证授权与登录态管理实现 25
4.3 基础数据管理实现 26
4.4 库存、批次、出入库实现 26
4.5 申领审批与调拨实现 26
4.5.1 申领审批实现 27
4.5.2 调拨实现与推荐机制 28
4.6 预警、补货建议与统计分析实现 30
4.7 日志、通知与事件管理实现 32
5 系统测试 34
5.1 测试环境与证据来源 34
5.2 自动化测试结果 34
5.3 业务场景验证 35

5.4 构建验证与问题边界 36
结束语 37
致　　谢 38
参考文献 39

1 绪论

1.1 研究背景

高校物资管理同时面对日常保障和突发事件两类场景：一方面，实验耗材、医疗防护、应急照明、清洁消杀等物资需要围绕部门申领、仓库出入库和批次追踪持续运转；另一方面，物资配置又必须服务于校园安全、公共卫生和大型活动保障等时效性任务。随着智慧校园建设逐步由网络基础设施扩展到业务协同层，传统依赖纸质台账、分散 Excel 表和人工电话确认的管理方式越来越难以支撑跨部门、跨校区的物资流转要求[1][2]。

从本项目代码与 SQL 结构可以看到，系统需要同时管理部门、角色、用户、校区、仓库、库位、分类、物资档案、库存、库存批次、申领单、调拨单、预警记录、通知消息和日志审计等多类对象。这些对象之间并不是孤立存在的：部门用户发起申领后需要经过审批人审核，审核通过后还要由仓库管理员结合库存与批次情况执行出库，最终再由使用方完成签收闭环；在调拨场景中，还存在调出仓与调入仓之间的库存协同、运输路径选择和状态流转问题。

当前仓库已经提供完整的前后端实现与测试代码，因此本科论文定稿不应停留在概念性叙述，而应回到真实仓库中的业务边界和实现证据。基于这一约束，本文把“校园物资智能管理系统设计与实现”界定为一个已经完成核心流程开发、具备自动化测试与前端构建验证基础的 B/S 架构项目，并围绕代码、SQL、配置和文档可以直接证明的事实展开分析与写作。

1.2 研究意义

本课题的意义首先不在于再描述一个抽象的“智慧校园愿景”，而在于把已经写进仓库的业务链条整理清楚。当前系统把申领、审批、出库、签收、调拨、预警和日志留痕拆成了明确的数据对象和状态节点，这些实现让校园物资管理从零散台账转向可追踪、可回溯的流程协同。

从应用角度看，项目的价值主要落在角色协作和责任确认上。SecurityConfig、AuthService 以及日志相关表说明，不同角色看到的菜单、能调用的接口和留下的操作记录都已经被明确区分。对校园场景来说，这种边界划分比单纯展示几个管理页面更有实际意义。

对论文写作本身，这一课题还有一个更直接的作用：它要求定稿回到仓库证据，而不是沿用旧稿里那些没有落到代码和配置上的结论。本轮改稿继续沿着这个方向处理，尽量把文字拉回实现现场，降低“看上去像论文模板”的痕迹。

1.3 国内外研究现状

1.3.1 国内研究现状

国内围绕智慧校园和高校信息化的研究已经积累了大量案例，常见切入点包括统一身份认证、流程电子化、后勤资源协同和数据共享平台建设[1]。在具体实现层面，基于 Spring Boot 与 Vue 的前后端分离方案已经成为高校业务系统中的常见组合，用于支撑角色隔离、列表查询、表单流转和基础统计展示[6][7][8][9]。

和本项目更接近的研究，多分布在图书、采购、资产和后勤物资等资源管理方向[4][5][10]。这些成果通常能够提供比较成熟的主数据维护、库存统计和审批流建模思路，但对批次追踪、临期处置和复杂状态迁移的讨论往往不够细，很多论文停留在功能截图层面。

从这一背景看，本项目的可取之处不在于提出了新的理论模型，而在于把校园物资管理里真正需要落地的对象和流程接上了代码实现。论文在此基础上展开分析，更容易把需求、设计和实现对应起来。

1.3.2 国外相关实践启示

国外公开资料里，与本项目最相关的并不是某一篇现成的校园物资论文，而是框架文档、工程规范和通用的业务系统实践。Spring Boot、Vue、Vite 和 JWT 相关文档提供的是实现路径：如何组织模块、如何处理认证、如何管理依赖、如何保持接口一致性[17][18][19][20]。

这类资料的共同特点是强调“先把主链路做实，再谈高级能力”。本项目在实现上也采取了类似思路，没有为了追求概念上的智能化去引入无法验证的大型平台依赖，而是先把登录鉴权、申领审批、库存批次、调拨推荐、预警生成和统计展示这些核心流程打通。

因此，本文对国外相关实践的吸收主要体现在工程方法上，而不是照搬一套宏大的系统叙事。这样的写法更符合当前仓库的真实状态，也更适合本科毕业设计的证据边界。

1.4 研究内容与论文结构

本文围绕当前仓库中的校园物资智能管理系统展开。正文先把角色分工、业务场景和功能边界讲清，再结合数据库结构、前后端依赖和统一响应机制说明总体设计，之后转入认证授权、基础数据、库存批次、申领审批、调拨执行、预警分析和日志通知等核心实现。

结构安排上，第 1 章处理选题背景、研究意义和相关实践；第 2 章回到需求与业务流程；第 3 章说明总体设计、数据库和接口安全；第 4 章展开关键实现；第 5 章只保留能够复核的测试与构建证据；最后在结束语中总结已完成工作和后续可扩展方向。

全文遵循同一条约束：只写当前仓库可以证明的事实。没有落到代码、SQL、配置或实际执行结果中的性能、部署、推广和运行成效，不在本文中当作结论使用。

2 系统需求分析

2.1 业务场景与角色划分

从 seed.sql、菜单装配逻辑和前端路由可以看出，系统真正围绕四类角色运转：系统管理员、仓库管理员、审批人和部门用户。管理员负责组织与主数据维护，仓库管理员处理入库、出库、批次和调拨执行，审批人处理审核动作，部门用户则承担申领、查看和签收这一侧的业务操作。

业务场景并不是平均铺开的。系统的主线首先是库存与批次管理，其次是部门申领到仓库执行的闭环流转，再往上才是跨仓调拨、预警生成和统计分析。这种层次关系与仓库表结构、服务拆分和页面入口是一致的，不是论文后加出来的抽象分类。

角色边界也不是只写在说明文档里。SecurityConfig、控制器上的 PreAuthorize 表达式和 AuthService.buildMenusByRole 共同决定了哪些用户能看到哪些菜单、调用哪些接口、进入哪些工作流页面，所以需求分析必须把这一点作为基本前提写清楚。

2.2 功能需求分析

2.2.1 基础数据管理需求

基础数据并不是陪衬模块。部门、角色、用户、校区、仓库、库位、分类、物资、供应商和系统配置这些对象一旦定义不稳，后续单据和统计都无法成立。前端已经为这些对象提供了独立页面，说明它们在系统里需要被长期维护，而不是一次性导入后就不再变化。

其中影响面最大的是物资档案。material_info 保存的编码、名称、分类、规格、单位、安全库存、保质期和供应商信息，会反复进入库存、预警、申领和补货建议等逻辑，所以这一部分的需求重点其实是主数据一致性，而不是单个表单页面本身。

2.2.2 库存与批次管理需求

校园物资管理不能只看一个总库存数字。inventory 负责汇总某物资在某仓库下的当前量和锁定量，inventory_batch 继续把来源批次、剩余数量、生产日期和到期日期拆开记录。这意味着系统在需求上必须同时支撑总账视图和批次视图，否则临期处理与先到期先出都无从谈起。

对应到操作层面，系统至少要能完成入库建批次、库存查询、按批次排序和出库扣减。InventoryService 里围绕批次排序和数量校验的实现也说明，这里需要的是可执行的业务规则，而不是一句笼统的“支持库存管理”。

2.2.3 申领审批与出库需求

申领审批是系统最核心的入口。apply_order 和 apply_order_item 采用主单加明细的结构，把申请部门、申请人、紧急等级、原因、场景和物资明细放进同一张业务单据里。围绕这张单据形成的状态链条覆盖了 DRAFT、SUBMITTED、APPROVED、REJECTED、OUTBOUND 和 RECEIVED 六个阶段，需求分析必须与这一真实状态模型保持一致。

代码里还能看到另一个校园场景：紧急申领。ApplyService 会根据 urgency_level 触发快速审批分支，所以系统不仅要服务于计划性领用，还要支撑临时保障、夜间处置和突发场景下更短的审批路径，同时留下审批意见和时间记录，避免流程缩短后失去追踪能力。

2.2.4 调拨协同需求

当一端仓库短缺、另一端仍有余量时，系统不能只让管理员手动改两个库存字段。transfer_order 和 transfer_order_item 表明，调拨必须先形成单据，再经历提交、审批、执行和签收，调出仓和调入仓的变化要在同一条业务语义下完成。

与普通出入库不同，调拨还要回答“从哪调、走哪条路径”这样的问题。项目里的 DijkstraUtil 和 transfer recommend 接口说明，系统已经把校区图上的距离计算接入到调拨场景中，所以这一部分的需求不只是单据流转，还包括来源仓选择的辅助支持。

2.2.5 预警、统计与支撑功能需求

要让校园物资系统长期可用，只把单据流转做通还不够。warning_record、notification、event_record、login_log 和 operation_log 这些表说明，预警、消息、事件和日志不是附属功能，而是支撑主链路运行的独立模块。库存不足、库存积压、临期、过期和异常领用等预警类型也都需要在需求阶段提前明确。

统计分析需求主要面向管理者。AnalyticsService 和 SmartService 表明，系统不仅要展示当前库存和单据状态，还要能基于历史出入库记录回答“最近哪些物资消耗快、哪些仓库存在风险、哪些物资需要补货”这类问题。也正因为如此，前面的主数据、库存和单据记录必须保持完整，否则分析结果没有基础。

2.3 非功能需求分析

非功能需求里最先落地的是安全与权限隔离。后端通过 Spring Security 过滤链、JWT、Refresh Token 和方法级权限控制保护接口访问，前端通过本地 token 存储、路由守卫和 401 处理维持登录态。换句话说，系统首先要保证不同角色只能进入各自职责范围内的页面和操作链路。

另一个现实要求是可维护性。仓库采用前后端分离组织方式，后端服务按认证、库存、申领、调拨、预警、智能分析和日志等模块拆分，数据库表统一保留 deleted、version、created_at 和 updated_at 一类通用字段。这种组织方式更像可持续演进的业务系统，而不是一次性演示项目。

系统还需要具备可测试性。当前仓库已经提供 H2 测试配置和一组围绕认证、申请、调拨、预警与异常处理的自动化测试类，所以论文可以把“可重复验证”当作一项真实非功能要求来写，而不是转向没有实测依据的并发和性能指标。

2.4 业务流程与用例分析

2.4.1 申领审批用例分析

在申领审批用例中，主要参与者包括部门用户、审批人和仓库管理员。部门用户首先创建申领单，录入申请原因、使用场景、紧急等级和物资明细；草稿单在提交前可以修改，提交后状态变为 `SUBMITTED`。审批人对单据进行审核，若不符合要求则退回为 `REJECTED` 并记录意见；若审核通过，则状态进入 `APPROVED`，仓库管理员据此执行出库。

出库环节并非单纯的状态修改，而是要结合库存与批次完成实物分配。系统根据库存和批次记录生成出库单与出库明细，回写申领明细的 `actual_qty`，并把申领单状态更新为 `OUTBOUND`。最终，部门用户对实际领用结果进行签收，状态更新为 `RECEIVED`，一张申领单才算闭环结束。

图 4-1 申领审批闭环流程图

2.4.2 调拨执行用例分析

调拨执行用例从仓库管理员发起单据开始。管理员根据目标仓库存量或保障需求创建调拨单，再结合推荐接口给出的候选来源仓完成提交。审批通过后，系统进入执行阶段，同时处理调出仓扣减、调入仓补入和状态推进，而不是只改动一个库存总量字段。

这类业务对一致性的要求比普通申领更高，因为一次调拨会同时影响两端仓库。目标仓完成签收前，系统不会把单据视为最终结束。通过这种设计，仓间转移被清楚地写成了申请、审核、执行、签收四段式链路。

2.4.3 预警处置与统计分析用例分析

预警处置用例由系统主动触发。WarningService.scan() 周期性检查库存和批次信息，生成库存不足、库存积压、临期、过期和异常领用等记录。管理员或仓库管理员在预警页查看待处理项后，可以补录处理说明并更新状态；若需要通知相关角色，系统还会写入消息表供后续查看。

统计分析则服务于管理决策。系统把出入库流水、库存分布和时间维度聚合到图表页与大屏页，并在智能分析模块中给出补货建议。它既是一个独立功能，也是前面主数据、库存和单据记录是否规范的反向检验。

3 系统总体设计

3.1 技术选型与总体架构

3.1.1 技术选型依据

结合仓库依赖文件可以确认，系统前端采用 Vue 3、Pinia、Element Plus、Axios、Vue Router 和 ECharts，使用 Vite 作为构建工具；后端采用 Spring Boot 3.3.5、Spring Security、MyBatis-Plus、JdbcTemplate 和 JWT；数据层使用 MySQL 8，并在测试阶段引入 H2 内存数据库。上述技术组合对应的是当前高校信息化项目中常见的前后端分离路线：前端负责页面交互与图表展示，后端负责认证授权、业务编排和数据持久化[10][11][12][13][14][16][17][18][19][20]。

选择 Vue 3 与 Pinia 的原因，在于项目存在较多角色菜单、表单状态和图表页面，需要较清晰的组件组织与轻量状态管理。结合 `package.json` 依赖范围与本次 `npm ls` 结果，可以确认当前前端实际安装的核心版本包括 Vue 3.5.29、Pinia 2.3.1、Vite 6.4.1、Element Plus 2.13.5 和 ECharts 5.6.0，这意味着前端工程已经使用组合式 API 与现代化构建链路。Element Plus 承担表单、表格、对话框、消息提示等通用界面元素，ECharts 提供库存与出入库统计图表的呈现能力[12][14][18][19]。

后端选择 Spring Boot 3 的主要原因，是其在 Web API、依赖管理、配置组织和测试支持方面较为成熟。Spring Security 用于认证与权限控制，MyBatis-Plus 负责实体映射和通用 CRUD，JdbcTemplate 用于统计与智能分析中的聚合 SQL。对于本科项目而言，这种组合既能覆盖典型企业级开发流程，又不会引入当前仓库无法证明的复杂中间件依赖，因此符合“功能完整、实现清晰、可测试”的设计目标[10][11][16][17][20]。

3.1.2 系统总体架构

从系统分层看，项目采用典型的 B/S 架构。浏览器作为统一访问入口，用户通过登录页进入系统后，根据角色获得不同菜单和页面；前端通过 HTTP/JSON 与后端接口交互；后端再按照控制层、服务层和数据访问层完成业务处理，并把结果写入 MySQL 数据库。对毕业论文而言，这种架构具有描述清晰、部署简洁和便于截图展示的优点。

在后端内部，控制层主要负责参数接收和权限约束，服务层负责状态流转、库存计算、批次分配和日志记录，数据访问层由 MyBatis-Plus Mapper 与 JdbcTemplate 共同构成。控制层与服务层之间通过统一响应对象 `ApiResponse{code,message,data}` 进行

对外封装，成功返回码固定为 `0`。这种统一返回机制使前端拦截器能够集中处理错误消息，减少页面端的重复判断。

在数据层面，系统并未采用高度拆分的微服务结构，而是使用单体后端配合模块化代码组织。结合当前仓库规模，这种方案更利于业务联调和测试验证：认证、库存、申请、调拨、预警和统计虽然属于不同功能域，但都围绕同一套用户、角色、仓库和物资主数据展开，因此在单体架构中更容易维持事务语义和数据一致性。

图 3-1 系统总体架构图

3.2 功能模块设计

根据前端路由、角色菜单和后端控制器，系统可以划分为八个相互关联的功能模块：认证与权限模块、基础数据模块、仓储库存模块、申领审批模块、调拨协同模块、预警与智能分析模块、统计分析模块、系统支撑模块。其中系统支撑模块又包括通知、事件、登录日志、操作日志和系统配置等功能，用于保障主业务的持续运行。

认证与权限模块负责用户登录、登出、刷新令牌、加载当前用户信息和动态菜单；基础数据模块负责用户、角色、部门、校区、仓库、库位、供应商、分类和物资档案维护；仓储库存模块负责入库、库存台账、批次追踪和出库执行；申领审批模块负责部门侧申领单创建、提交、审批和签收；调拨协同模块负责跨仓调拨推荐、审批、执行和签

收。这样的划分既贴合代码目录，也符合校园物资业务从“主数据准备”到“单据闭环执行”的实际顺序。

预警与智能分析模块、统计分析模块和系统支撑模块则体现了项目的管理深度。预警与智能分析模块提供定时预警、需求预测和补货建议；统计分析模块基于业务数据生成图表和大屏视图；系统支撑模块通过消息、日志与事件记录提升可追溯性。整体来看，系统功能并非停留在 CRUD，而是围绕校园物资“计划、执行、监督、分析”的完整闭环展开。

图 3-2 系统功能模块图

3.3 数据库设计

3.3.1 数据库设计原则

数据库设计以“主数据统一、业务单据分层、审计信息独立”为基本原则。主数据层负责保存组织结构、角色、用户、仓库、物资和供应商等静态或低频变化信息；业务单据层负责记录申领、出入库、调拨和预警；审计与支撑层负责保存日志、通知和事件等辅助信息。这样的分层方式可以避免把所有业务语义压缩到少数宽表中，便于后续统计查询和模块拆分。

从字段设计上看，多数业务表都包含 `deleted`、`version`、`created_at` 和 `updated_at` 字段，体现了逻辑删除、乐观锁预留和时间追踪的统一思想。对于库存、批次、单据、预警和日志等需要长期保留历史记录的场景，逻辑删除比物理删除更适合审计与回溯；统一时间字段则为统计分析、预警扫描和测试数据初始化提供了时间维度基础。

索引设计主要围绕业务查询入口展开。例如 `inventory` 表在物资与仓库上建立联合唯一约束，确保同一物资在同一仓库只有一条总库存记录；`inventory_batch` 表在到期日期上建立索引，便于按批次进行临期查询；申领单、调拨单、预警和日志等表也围绕状态、人员或时间字段建立索引，以支持列表页和统计查询的主要过滤条件。

3.3.2 实体关系设计

从实体关系角度看，本系统可以拆分为三组核心子域。第一组是 RBAC 与组织子域，包括部门、角色、用户、刷新令牌和日志，它决定了谁可以访问系统、可以看到哪些菜单以及关键操作由谁执行。第二组是库存与批次子域，包括分类、物资档案、仓库、库存总表、库存批次和出入库记录，它决定了库存总量与批次明细如何被维护。第三组是业务单据与支撑子域，包括申领单、调拨单、预警记录、通知和事件，它负责把具体业务动作串联为闭环。

这些子域之间存在清晰的主外键语义。部门用户与审批人都来自 `sys_user`，而 `sys_user` 又通过 `dept_id` 和 `role_id` 关联组织结构；库存和批次通过 `material_id` 与 `warehouse_id` 同时指向物资和仓库；申领单、调拨单和预警记录则分别通过用户、部门、仓库和物资标识与主数据连接。虽然当前 SQL 没有在所有表上显式声明外键约束，但从字段命名和服务调用看，实体关系是稳定且明确的。

图 3-3 RBAC 与组织实体图

图 3-4 库存与批次实体图

图 3-5 业务单据、预警与通知实体图

3.3.3 关键数据表设计

为恢复原稿中较完整的数据库字段说明方式，同时避免继续沿用旧表名和旧字段名，本节保留 current working draft 已确定的 12 张关键表范围，统一改为详细字段说明表。表格的展示粒度参考原稿数据库章的详表版式，但字段内容、约束和说明均以当前 schema.sql 为唯一依据。这样处理既补回了数据库章缺失的信息密度，也避免把已经退出现仓库的旧结构重新写回正文。

表 3-1 用户信息表（sys_user）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
username	VARCHAR	50	唯一，非空	登录账号
password	VARCHAR	255	非空	BCrypt 密码串
real_name	VARCHAR	100	非空	姓名
dept_id	BIGINT	-	可空	关联部门
role_id	BIGINT	-	可空	关联角色
status	TINYINT	-	非空，默认 1	当前状态标记
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 3-2 物资档案表（material_info）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
material_code	VARCHAR	50	唯一，非空	物资编码
material_name	VARCHAR	100	非空	物资名称
category_id	BIGINT	-	可空	所属分类
spec	VARCHAR	100	可空	规格型号
unit	VARCHAR	20	可空	计量单位
safety_stock	INT	-	非空，默认 0	安全库存阈值
shelf_life_days	INT	-	可空	保质期天数
supplier	VARCHAR	255	可空	供应商名称
unit_price	DECIMAL	10,2	可空	单价
remark	VARCHAR	255	可空	备注信息
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 3-3 仓库信息表（warehouse）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
warehouse_name	VARCHAR	100	非空	仓库名称
campus	VARCHAR	100	可空	所属校区
address	VARCHAR	255	可空	地址
manager	VARCHAR	100	可空	负责人
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 3-4 库存总表（inventory）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
material_id	BIGINT	-	非空	关联物资
warehouse_id	BIGINT	-	非空	关联仓库
current_qty	INT	-	非空，默认 0	当前库存数量
locked_qty	INT	-	非空，默认 0	锁定数量
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 3-5 库存批次表（inventory_batch）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
material_id	BIGINT	-	非空	关联物资

warehouse_id	BIGINT	-	非空	关联仓库
batch_no	VARCHAR	100	非空	批次号
in_qty	INT	-	非空，默认 0	入库数量
remain_qty	INT	-	非空，默认 0	剩余数量
production_date	DATE	-	可空	生产日期
expire_date	DATE	-	可空	到期日期
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 3-6 申领单主表（apply_order）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
dept_id	BIGINT	-	非空	关联部门
applicant_id	BIGINT	-	非空	申请人
urgency_level	INT	-	非空，默认 0	紧急等级
status	VARCHAR	30	非空	当前状态标记
reason	VARCHAR	255	可空	业务原因
scenario	VARCHAR	255	可空	使用场景
fast_track	TINYINT	-	非空，默认 0	快速审批标记
approver_id	BIGINT	-	可空	审批人
approve_remark	VARCHAR	255	可空	审批意见
approve_time	DATETIME	-	可空	审批时间
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 3-7 申领单明细表（apply_order_item）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
apply_order_id	BIGINT	-	非空	关联申领单
material_id	BIGINT	-	非空	关联物资
apply_qty	INT	-	非空	申领数量
actual_qty	INT	-	非空，默认 0	实际出库数量
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 3-8 出库单主表（stock_out）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识

apply_order_id	BIGINT	-	可空	关联申领单
warehouse_id	BIGINT	-	非空	关联仓库
operator_id	BIGINT	-	可空	操作人
remark	VARCHAR	255	可空	备注信息
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 3-9 调拨单主表（transfer_order）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
from_warehouse_id	BIGINT	-	非空	调出仓库
to_warehouse_id	BIGINT	-	非空	调入仓库
status	VARCHAR	30	非空	当前状态标记
reason	VARCHAR	255	可空	业务原因
applicant_id	BIGINT	-	可空	申请人
approver_id	BIGINT	-	可空	审批人
approve_remark	VARCHAR	255	可空	审批意见
approve_time	DATETIME	-	可空	审批时间
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 3-10 预警记录表（warning_record）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
warning_type	VARCHAR	50	非空	预警类型
material_id	BIGINT	-	可空	关联物资
warehouse_id	BIGINT	-	可空	关联仓库
content	VARCHAR	255	非空	内容描述
handle_status	VARCHAR	30	非空，默认 UNHANDLED	处理状态
handler_id	BIGINT	-	可空	处理人
handle_remark	VARCHAR	255	可空	处理说明
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 3-11 通知消息表（notification）

字段名	数据类型	长度	约束	注释描述

id	BIGINT	-	主键，自增	主键标识
title	VARCHAR	200	非空	标题
content	VARCHAR	500	非空	内容描述
msg_type	VARCHAR	50	非空，默认 SYSTEM	消息类型
target_user_id	BIGINT	-	可空	目标用户
is_read	TINYINT	-	非空，默认 0	是否已读
biz_type	VARCHAR	50	可空	业务类型
biz_id	BIGINT	-	可空	业务编号
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

表 3-12 登录日志表（login_log）

字段名	数据类型	长度	约束	注释描述
id	BIGINT	-	主键，自增	主键标识
user_id	BIGINT	-	可空	关联用户
username	VARCHAR	50	可空	登录账号
login_ip	VARCHAR	100	可空	登录 IP
login_status	VARCHAR	20	非空，默认 SUCCESS	登录状态
login_time	DATETIME	-	非空，默认当前时间	登录时间
user_agent	VARCHAR	500	可空	客户端标识
remark	VARCHAR	255	可空	备注信息
deleted	TINYINT	-	非空，默认 0	逻辑删除标记
version	INT	-	非空，默认 0	版本号
created_at	DATETIME	-	非空，默认当前时间	创建时间
updated_at	DATETIME	-	非空，默认当前时间	更新时间

3.4 接口与安全设计

3.4.1 接口设计

接口设计不是抽象的 REST 口号，而是前后端协作的具体约束。当前项目统一以 /api 作为业务前缀，再按认证、库存、申领、调拨、预警、统计、事件和消息等功能域拆分接口，这一结构已经能从控制器分布和前端请求封装中直接看到。

更关键的是统一响应外壳。frontend/src/api/http.js 的拦截逻辑表明，所有业务接口都围绕 ApiResponse 的 code、message 和 data 三个字段组织返回结果，且前端把 code 等于 0 视为成功。这种一致性减少了页面侧的分支判断，也让异常处理可以回到同一出口。

3.4.2 认证与授权设计

认证链路由登录接口、令牌签发、刷新令牌续期和前端重放机制共同组成。JwtTokenProvider 负责生成和校验访问令牌，AuthService 负责登录、获取当前用户与菜单装配，http.js 则在 401 场景下触发刷新接口并重放原请求。这说明当前系统已经把登录态管理落实为一条完整链路，而不是简单地在浏览器里存一个长期有效字符串。

授权控制分布在两个层面。其一是 SecurityConfig 打开的方法级安全能力和控制器上的 PreAuthorize 表达式，其二是 AuthService.buildMenusByRole 返回的菜单集合。接口层和导航层一起工作后，系统管理员、仓库管理员、审批人和部门用户才真正表现出不同的页面与操作边界。

3.4.3 安全与审计设计

在安全细节上，当前仓库已经落地的是密码加密、跨域控制、刷新令牌清理和审计留痕。用户密码采用 BCrypt 存储，刷新令牌以哈希形式写入 auth_refresh_token 表，并通过定时任务清理过期或撤销记录；登录日志和操作日志则为关键动作提供了追踪基础。

需要收束的是结论边界。当前项目没有独立的压测报告、安全扫描记录或生产部署监控，所以论文中的“安全设计”只讨论已经实现的访问控制、凭证管理和操作审计，不把高并发防护、网关治理或上线效果写成既成事实。

4 系统详细设计与实现

4.1 开发环境与运行环境

当前项目使用前后端分离开发方式。后端基于 Maven 构建，依赖由 `backend/pom.xml` 管理；前端基于 Node.js 与 Vite 构建，依赖由 `frontend/package.json` 管理。项目文档 `DEPLOY.md` 给出了本地运行和构建步骤，测试环境则通过 `backend/src/test/resources/application-test.yml` 把数据源切换为 H2 内存数据库，以便在不依赖外部 MySQL 服务的前提下执行自动化测试。

从实现角度看，开发环境与运行环境并未刻意追求复杂化。系统在 JDK 17、Maven 3.8+、Node.js 18+ 和 MySQL 8 环境下即可完成开发、测试和构建；前端通过 Vite 提供开发服务器与生产打包，后端通过 Spring Boot 提供接口服务与调度任务。这种环境配置符合课程设计项目“小而完整”的特点，有利于在毕业答辩前完成部署复现和运行截图采集。

表 4-1 系统开发与运行环境

项目	版本/配置	说明
JDK	17	后端运行环境，`pom.xml` 中声明 `java.version=17`
Maven	3.8+	后端依赖管理与打包工具
Spring Boot	3.3.5	后端基础框架
MyBatis-Plus	3.5.7	ORM 与通用数据访问框架
Node.js	18+	前端构建环境
Vue	3.5.29	前端核心框架，本次 `npm ls` 可见已安装版本
Vite	6.4.1	前端开发与构建工具，本次构建输出显示版本
Element Plus	2.13.5	前端组件库，本次 `npm ls` 可见已安装版本
ECharts	5.6.0	统计图表组件
MySQL	8.0	业务数据持久化存储
H2	内存模式	测试阶段数据源

4.2 认证授权与登录态管理实现

认证实现从 /api/auth/login 进入，但关键并不只是“用户输入账号密码”。前端 auth store 在登录成功后会立即拉取当前用户资料与动态菜单，说明登录动作和角色可见范围是一次连续初始化过程。对论文而言，这比单独介绍某个登录表单更能说明系统的真实运行方式。

登录态维持主要依赖 frontend/src/api/http.js 的请求与响应拦截器。请求阶段统一附加 Bearer Token，响应阶段先检查 ApiResponse 的 code，再在收到 401 时进入刷新流程。脚本里维护的等待队列让多个并发请求不会重复刷新令牌，这一点直接对应了仓库中的实现细节。

后端与之配合的是 JwtTokenProvider、AuthService 和 auth_refresh_token 表。访问令牌负责短期身份识别，刷新令牌负责续期与撤销，清理任务再把过期记录回收掉。这样形成的不是概念上的 JWT 介绍，而是一条已经写进代码和数据表的登录、续期、退出链路。

为了直观展示系统登录入口与统一认证界面，本节补充运行截图如图 4-3 所示。

图 4-3 系统登录界面

4.3 基础数据管理实现

基础数据管理模块覆盖用户、角色、部门、校区、仓库、库位、供应商、分类和物资档案等对象。前端针对这些对象分别提供列表、查询、新增、编辑和删除页面，后端则以相对统一的控制器和服务模式组织 CRUD 逻辑。虽然这些页面的交互形式类似，但在系统中承担的作用并不相同：部门和角色决定权限边界，仓库与库位决定库存归属，供应商和物资档案则决定入库、预警和补货分析的主数据来源。

物资档案模块是基础数据中的关键节点。物资编码、名称、分类、规格、单位、安全库存、保质期、供应商和单价等信息不仅用于列表展示，还会被库存预警、出库扣减和补货建议等后续逻辑直接使用。因此，物资档案模块的实现重点不在于界面表单本身，而在于通过统一主数据避免“同一物资多种命名、不同仓库重复维护”的问题。

基础数据模块还承担初始化业务场景的作用。`seed.sql` 为管理员、仓库管理员、审批人和部门用户准备了基础账号和样例数据，这使系统在开发与测试阶段能够快速进入可操作状态。对论文写作而言，这一事实意味着系统并非停留在孤立的接口层，而是具备了可以支撑完整业务链路的初始数据环境。

4.4 库存、批次、出入库实现

库存管理模块的核心是同时维护库存总量与批次明细。入库时，系统不仅更新 `inventory` 表中的 `current_qty`，还在 `inventory_batch` 中写入批次号、入库量、剩余量、生产日期和到期日期，从而保证总库存和批次账保持一致。出库时则反向根据批次顺序扣减剩余量，再同步更新总库存。这样设计的直接好处，是可以在总量统计与批次追溯之间取得平衡。

`InventoryService` 的实现体现了“按到期日优先”的业务规则。系统查询指定物资在某仓库下所有剩余数量大于 0 且未过期的批次，按 `expire_date` 和主键升序排列后依次扣减。该策略既能优先消耗临近到期物资，降低损耗风险，也方便论文在实现章节中把“批次推荐”解释为一项已落地的业务规则，而不是停留在先进先出原则的口头描述。

出入库模块还负责把库存变化与单据状态关联起来。入库单负责建立新批次并增加库存，出库单则通常与申领单关联，在完成拣货后回写申领明细实际数量并推动申领单进入 `OUTBOUND` 状态。也就是说，库存模块并不是孤立计算数量变化，而是在业务上下文中完成库存扣增、批次更新和状态推进的联动。

4.5 申领审批与调拨实现

4.5.1 申领审批实现

申领审批实现围绕 `ApplyService` 展开。创建申领单时，系统先插入主表 `apply_order`，初始化状态为 `DRAFT`，再批量插入明细表 `apply_order_item`。提交时若单据仍处于草稿状态，则进一步根据 `urgency_level` 判断流转分支：普通单据进入 `SUBMITTED` 等待审批，紧急等级大于等于 2 的单据则直接置为 `APPROVED`，同时记录审批人、审批意见和审批时间。这一逻辑比传统论文中常见的“提交后统一待审”更贴近真实业务约束。

审批通过、驳回和签收等动作都由明确的方法处理，并通过 `OrderStatus` 常量约束状态机边界。例如，只有 `SUBMITTED` 状态的单据可以被审批或驳回，只有 `OUTBOUND` 状态的单据才允许签收。每个关键动作完成后，系统都会调用 `OperationLogService` 写入操作日志，从而形成业务动作与审计记录的同步留痕。

申领审批模块的实现价值在于，它把角色协同、单据明细、状态机和库存执行串成了一条完整链路。部门用户关注的是“能否快速发起申请”，审批人关注的是“是否符合领用条件”，仓库管理员关注的是“如何准确出库并回写数量”，而系统通过主表、明细表和日志表把这些差异化需求统一到同一套数据模型下。

部门用户发起物资申请时，可以在申领页面直接录入原因、场景和物资明细，运行界面如图 4-4 所示。

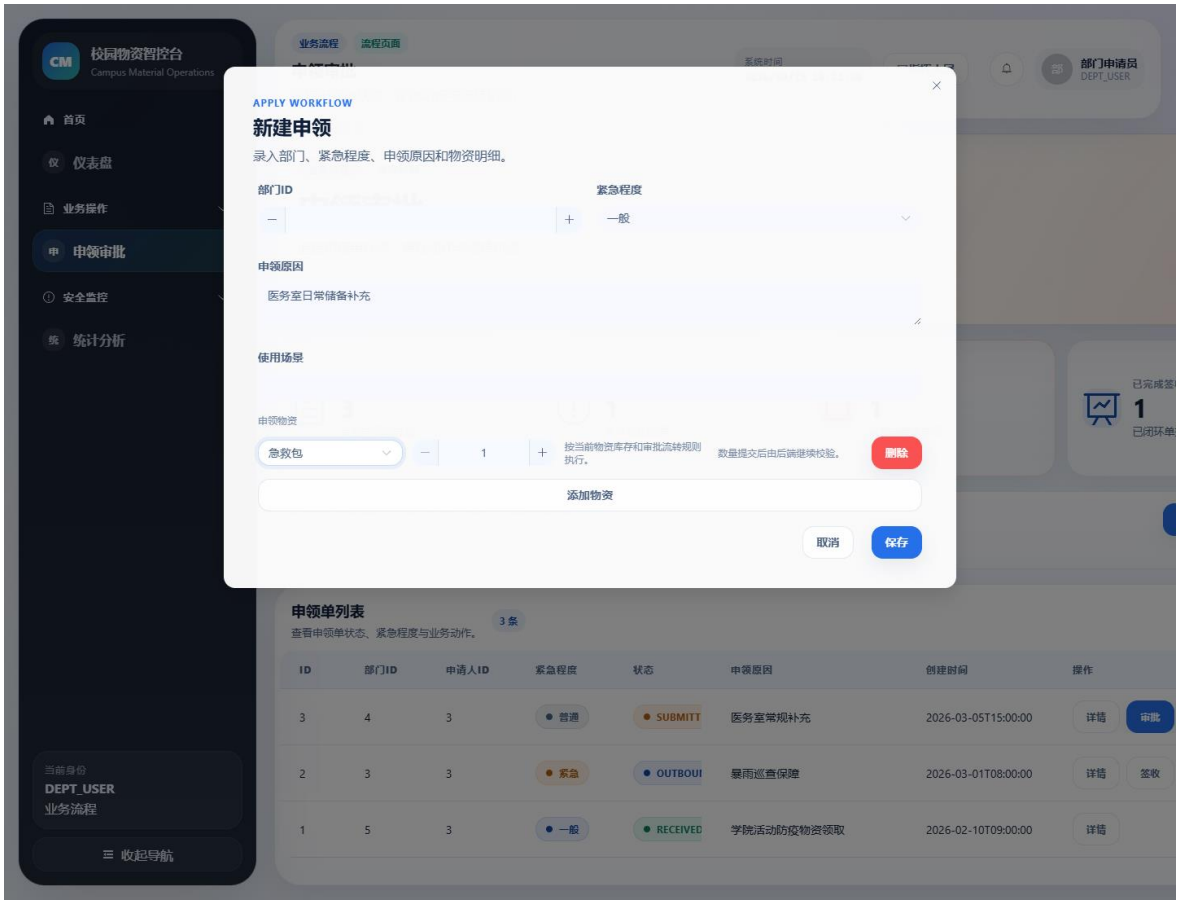

图 4-4 部门用户申领界面

审批人与仓库管理员分别在审批列表和出库页面完成业务闭环，组合运行界面如图 4-5 所示。

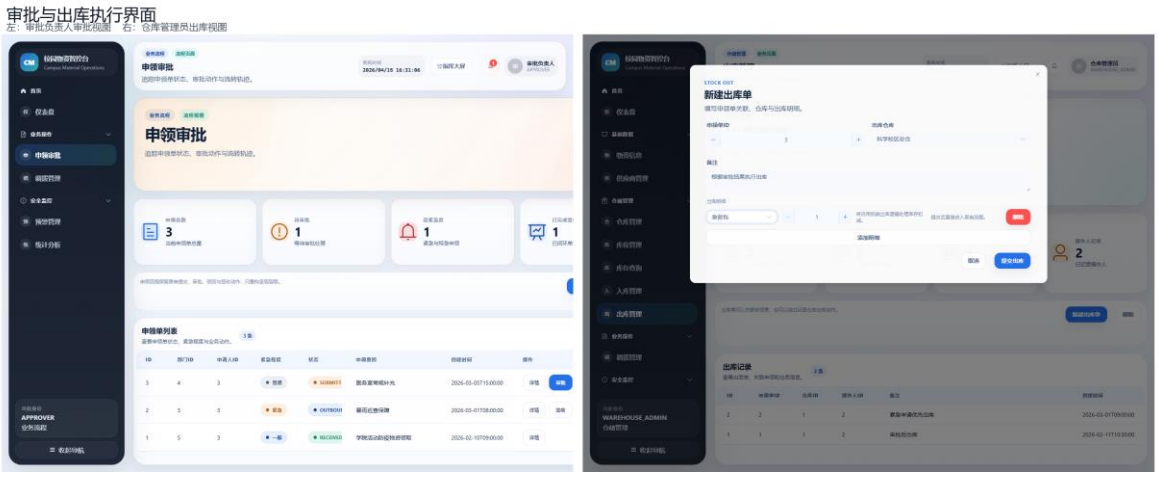

图 4-5 审批与出库执行界面

4.5.2 调拨实现与推荐机制

调拨实现与申领类似，也采用主表加明细表模式，但其复杂度更高。`TransferService.create()` 在校验调出仓与调入仓不能相同后，生成 `DRAFT` 状态的调

拨主单和多条明细；提交、审批、驳回、执行与签收则分别由独立方法处理。执行阶段系统不仅要检查调出仓库存是否充足，还要逐批次扣减来源批次，并在调入仓生成新的批次记录，这使调拨真正成为一次“跨仓库存迁移”。

调拨推荐机制建立在 `DijkstraUtil` 的预设校区图之上。服务层调用 `calculateShortestPaths(targetCampus)` 计算目标校区到其他节点的最短距离，再结合库存表筛选“库存数量足够”的候选仓库，并按距离从小到大排序返回。这里的“智能”并不是复杂地图服务，而是基于预设节点和边权的最短路径计算，适合作为校园内部多校区、多仓点场景下的轻量辅助决策能力。

这一实现说明系统对“跨仓协同”的理解已经超出普通单仓库存管理：仓库之间不是互不关联的孤岛，而是可以围绕库存余量、距离和审批流程进行协同。论文在此处需要强调的是推荐逻辑的边界，即它解决的是候选仓排序问题，而不是完整的物流调度优化问题，从而保持与代码实现一致。

图 4-2 调拨执行与推荐流程图

仓库管理员在调拨页面可直接发起跨仓调拨，并调用推荐结果辅助选择来源仓库，运行界面如图 4-6 所示。

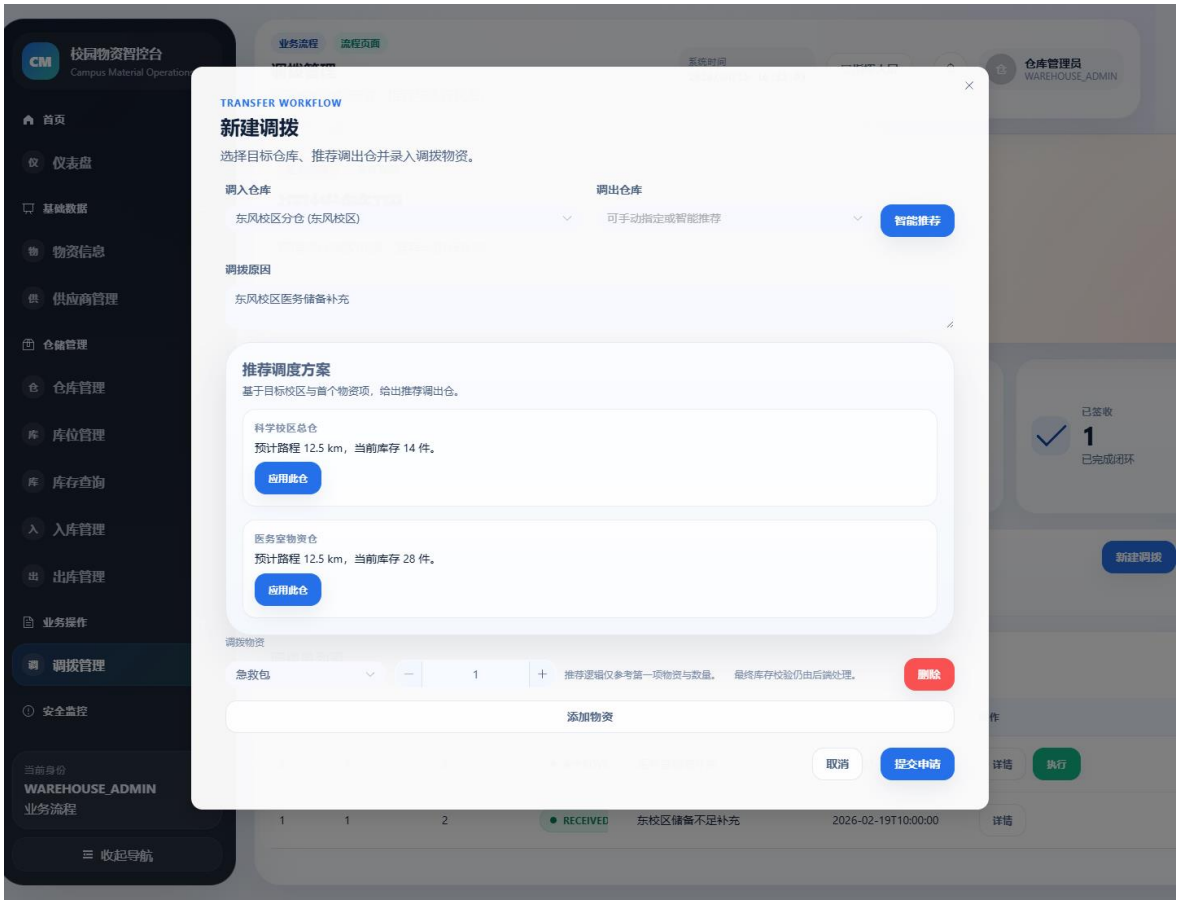

图 4-6 调拨推荐界面

4.6 预警、补货建议与统计分析实现

预警模块由 `WarningService` 负责实现，核心入口是带有 `@Scheduled(cron = "0 0/30 * * * ?")` 注解的 `scan()` 方法。系统每 30 分钟扫描一次库存与批次数据，分别执行低库存、库存积压、临期、过期和异常领用五类检测逻辑，并在不存在相同未处理预警时创建新的 `warning_record`。这一定时扫描机制让预警成为主动发现问题的能力，而不是完全依赖人工检查列表。

五类预警的判定依据都能在代码中找到明确来源：库存不足与库存积压依赖 `material_info.safety_stock`；临期与过期依赖批次到期日期；异常领用则比较近 7 天出库总量与近 30 天折算周均值的关系。需要特别说明的是，当前实现并未读取 `system_config` 中的阈值配置，因此论文不能将其写成“系统配置动态驱动预警阈值”，而应准确描述为基于安全库存和固定窗口规则的扫描机制。

智能分析模块由 `SmartService` 提供两个能力：一是读取近 6 个月的出库历史，计算平均值并生成未来若干月预测；二是按近 30 天出库量估算日均消耗，结合安全库存和保障天数计算目标库存与建议补货量。这种实现方式与文献中常见的移动平均思想

保持一致，但工程上采用的是简单、可解释、可落地的平均值计算，而不是论文中经常被泛化描述的复杂预测模型[15]。

统计分析则由 `AnalyticsService` 通过聚合 SQL 实现。系统围绕月度出入库趋势、物资分布、预警数量等维度生成图表数据，并由前端图表页和大屏页负责展示。由于统计结果直接依赖业务表中的真实记录，因此其价值不只是“展示漂亮图表”，更在于帮助管理员观察物资流动规律、识别高频消耗物资并辅助补货决策。

预警模块在列表中同时展示预警类型、处理状态和处置入口，真实运行界面如图 4-7 所示。

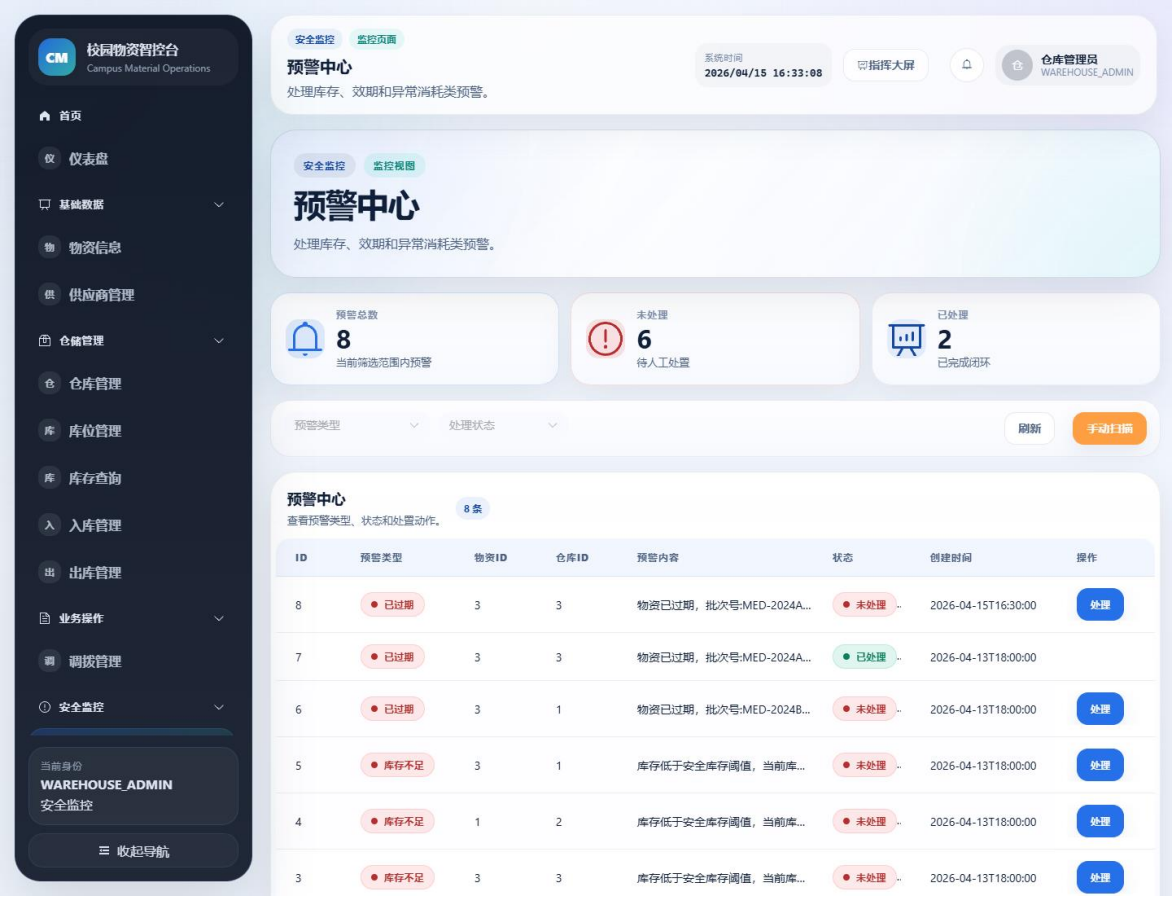

图 4-7 预警管理界面

统计分析模块通过图表卡片集中展示库存、趋势和排行结果，真实运行界面如图 4-8 所示。

图 4-8 统计分析界面

4.7 日志、通知与事件管理实现

日志、通知和事件管理模块承担的是“让业务过程可追踪”的职责。登录日志记录用户账号、登录 IP、登录结果、时间和客户端标识，操作日志记录操作人、业务模块、操作类型和明细内容，两者共同构成对关键行为的审计基础。由于申领、审批、出库、调拨、预警处理等操作都会写入日志，系统能够在后续排查中回溯“谁在什么时候做了什么”。

通知模块以 `notification` 表为中心，保存标题、内容、消息类型、目标用户、已读状态及关联业务信息。它的实现价值在于把业务结果从“页面即时提示”扩展为“可再次查看的消息记录”，从而适应校园场景中审批、预警、事件等信息需要跨时间查看的需求。部门用户、审批人和仓库管理员可以在自身角色页面中查看与自己相关的消息，提升协同效率。

事件管理模块则进一步扩展了系统对校园保障场景的覆盖面。`event_record` 表可以记录事件标题、类型、等级、位置、上报人、处理结果和关闭时间，说明项目已经考虑到校园突发事件与物资管理之间的联动关系。虽然当前论文不把它写成完整应急指挥

平台，但将事件管理作为支撑模块纳入实现，能够体现系统对校园综合保障业务的延展能力。

5 系统测试

5.1 测试环境与证据来源

本章只采用当前仓库可以重复执行的验证材料，包括后端自动化测试代码、H2 测试配置、Maven 测试结果和前端生产构建结果。这样写的目的很直接：把论文的测试结论限制在可以复核的范围内，不再沿用旧稿里那些无法复现的压测和上线描述。

后端测试环境由 backend/src/test/resources/application-test.yml 指向 H2，并结合测试资源目录中的 schema 初始化数据结构。这意味着关键业务逻辑可以在不依赖外部 MySQL 实例的情况下重复执行，测试结论也更容易回到仓库环境中核验。

在本轮定稿过程中，2026 年 4 月 14 日已经实际执行过 mvn -f backend/pom.xml test 与 npm --prefix frontend run build，两条命令分别得到后端 42 项测试通过、前端生产构建成功的结果。第 5 章的核心证据就建立在这两次真实执行之上。

为了给第 4 章提供界面佐证，前一轮还额外使用了 screenshot 隔离 profile 和最小演示数据采集运行截图。该环境只服务于论文取证，不替代本文对主项目实现边界的判断。

5.2 自动化测试结果

从测试类分布看，当前自动化测试没有试图覆盖一切，而是集中在最容易出错的主链路上：统一响应与异常处理、认证与令牌管理、申领业务、调拨业务和预警接口都已有对应测试。这种覆盖方式更接近课程设计项目的现实做法，即优先把状态流转和权限校验这些高风险点守住。

2026 年 4 月 14 日执行 Maven 测试后，后端共通过 42 项测试。这个结果至少说明当前版本在核心业务逻辑、配置加载和关键断言层面保持了可执行状态，也说明测试结论并不依赖人工预置数据库环境。

表 5-1 后端自动化测试执行情况

测试类别	对应测试类/范围	验证重点	结果
统一响应与异常处理	ApiResponseTest、BizExceptionTest、GlobalExceptionHandlerTest	统一返回结构、异常封装与错误消息	2026-04-14 执行通过
认证与令牌管理	AuthServiceTest、JwtTokenProviderTest、AuthRefreshTokenCleanupTaskTest	登录、JWT、刷新令牌清理与菜单装配	2026-04-14 执行通过
申领业务	ApplyServiceTest、ApplyControllerTest	申领单创建、提交、审批和接口访问	2026-04-14 执行通过
调拨业务	TransferServiceTest、TransferControllerTest	调拨单创建、审批、执行和推荐接口	2026-04-14 执行通过
预警接口	WarningControllerTest	预警列表与处理接口	2026-04-14 执行通过
总体结果	共 11 个测试类、42 项测试	核心模块可执行性	全部通过

5.3 业务场景验证

除自动化测试外，论文还可以基于代码、路由和 seed.sql 样例数据对典型业务场景做可复验说明。这里强调的不是“所有场景都已真实上线运行”，而是主链路已经具备清楚的入口、状态变化和落库对象，后续答辩演示可以按这些线索逐项核对。

结合当前实现，至少有三类场景已经在代码层形成闭环：部门用户提交申领单后经审批人与仓库管理员流转直至签收；仓库管理员创建调拨单并结合推荐接口完成跨仓执行；系统按周期生成预警并在统计与智能分析模块中输出管理视角结果。这样归纳比旧稿中那类“流程非常流畅、体验良好”的结论更接近可验证事实。

表 5-2 典型业务场景验证说明

业务场景	主要入口	关键证据	说明
普通申领审批与签收	/apply/list	ApplyService、InventoryService、OrderStatus	支持 DRAFT→SUBMITTED→APPROVED→OUTBOUND→RECEIVED 流转
紧急申领快速审批	/apply/list	ApplyService.submit()	urgency_level>=2 时直接审批通过并记录审批信息
跨仓调拨推荐与执行	/transfer/list	TransferService、DijkstraUtil	支持候选仓排序、审批、批次迁移与签收
预警扫描与处理	/warning/list	WarningService.scan()、warning_record	支持五类预警生成与处理状态更新
补货建议与趋势分析	/analytics/charts、/bigscreen	SmartService、AnalyticsService	支持近 6 个月历史统计、预测与建议补货量计算
日志与通知追踪	/log/*、/notification/list	login_log、operation_log、notification	支持对关键业务动作的审计与消息留存

5.4 构建验证与问题边界

前端构建验证来自 2026 年 4 月 14 日执行 npm --prefix frontend run build 的实际结果。构建成功说明当前仓库中的 Vue 页面、路由、组件依赖和打包配置仍然保持一致，没有出现阻断交付的基础构建错误。

同样需要明确边界。现有测试与构建结果只能证明“项目可执行”“核心逻辑在现有覆盖下通过验证”，不能外推到高并发稳定性、生产网络部署效果和长期运维可靠性。仓库里没有对应的压测报告、监控数据或生产日志，所以这些结论继续不写入正文。

从论文收口角度看，后续工作主要剩下 Word 侧的目录、题注和分页复核，以及参考文献格式校对。这些属于排版与展示层面的完善，不改变本文已经建立的事实边界。

结束语

本文围绕当前项目仓库，对校园物资智能管理系统的需求、总体设计、关键实现和测试证据进行了重新梳理。本轮改稿继续压缩旧稿中偏模板化、偏结论化的表达，并把数据库章和测试章重新拉回到代码、SQL、配置和执行结果可以直接证明的范围内。

从现有实现看，系统已经完成了认证授权、基础数据维护、库存与批次管理、申领审批、调拨协同、预警扫描、补货建议、统计分析、通知消息、事件记录和日志审计等核心能力。它的价值不在于宣称形成了多大规模的平台，而在于主业务链路已经有了可以运行、可以测试、可以说明的数据结构和服务逻辑。

结合 2026 年 4 月 14 日后端 42 项测试通过和前端生产构建成功的事实，可以把当前项目视为一个边界清楚、证据完整度较高的本科毕业设计成果。后续若继续完善，也应沿着现有实现往前推进，而不是重新引入无法验证的宏大叙事。

致　　谢

本文的完成离不开指导教师在选题、实现和论文修改过程中的持续指导，也离不开项目仓库中已有代码、测试、配置与文档材料提供的直接支撑。在定稿过程中，针对旧草稿中存在的事实偏差、结构冗余和证据不足问题，能够重新回到真实项目资产完成梳理，对本人理解软件工程类毕业设计“以代码为证据”的写作方式具有重要帮助。

同时，也感谢在项目开发与论文撰写期间给予支持的同学和家人。正是在反复调试、测试、修订和校对的过程中，才逐步形成了当前这一版更贴近仓库实现、结构更完整、表述更审慎的毕业论文工作稿。

参考文献

[1] 陈志敏, 何建国. 大数据背景下智慧校园建设与管理模式创新探讨[J]. 现代教育管理, 2020, 12(8): 45-51.

[2] 张强, 李红梅. 高校突发公共事件应急物资储备体系构建研究[J]. 中国安全生产科学技术, 2022, 18(3): 67-73.

[3] 冯绍彬, 周华. 基于角色的动态权限管理系统的应用设计[J]. 郑州轻工业大学学报(自然科学版), 2020, 35(4): 71-76.

[4] 刘丹阳, 杨华, 张诗桐. 基于 Django 的个性化图书推荐管理系统设计与实现[J]. 山西电子技术, 2025(03): 64-66,80.

[5] 岑丹. 基于人工智能的图书馆信息管理系统设计与实现[J]. 吉林大学学报(信息科学版), 2025, 43(06): 1346-1351.

[6] 李淼淼, 季节, 李坡, 等. 基于 SpringBoot 和 Vue 3 的移动学习管理系统的设计与实现[J]. 无线互联科技, 2026, 23(04): 81-86.

[7] 杨茜, 吴加莹. 基于 B/S 架构的高校教务管理系统设计与实现[J]. 电脑知识与技术, 2026, 22(06): 58-60,76.

[8] 崔靖茹, 文华, 刘宏磊, 等. 基于 Vue 和 SpringBoot 框架的高校信息化项目管理系统的设计与实现[J]. 现代信息科技, 2025, 9(22): 77-81.

[9] 薛楠楠, 张伟, 钟化雨, 等. 基于 SpringBoot+Vue 的事故报告数据库系统设计与实现[J]. 计算机仿真, 2025, 42(07): 456-462,570.

[10] 王伟, 刘明. 基于 Spring Boot 的后端架构设计与性能优化研究[J]. 计算机系统应用, 2021, 30(5): 102-109.

[11] 刘强, 周波. MyBatis-Plus 企业级高可用开发指南[M]. 北京: 电子工业出版社, 2021: 89-105.

[12] 李东, 赵莉. Vue3+Vite 开发实战: 组合式 API 与工程化实践[M]. 北京: 人民邮电出版社, 2023: 155-180.

[13] 李晓东. MySQL 8.0 性能调优实战教程[M]. 北京: 清华大学出版社, 2022: 24-39.

[14] 赵玉明, 林晓. ECharts 在 Web 报表数据可视化中的应用分析[J]. 信息技术与信息化, 2021(4): 21-25.

[15] 孙阳, 陈洁. 一种基于移动平均预测模型的库存预警系统设计[J]. 物流工程与管理, 2022, 44(2): 33-38.

[16] 杨俊. 新一代 Spring Security 原理与 JWT 无状态鉴权探索[D]. 武汉: 武汉科技大学, 2020.

[17] Spring Team. Spring Boot Reference Documentation[EB/OL]. https://docs.spring.io/spring-boot/docs/3.3.5/reference/html/, 2026-04-14.

[18] Vue Team. Vue.js Documentation[EB/OL]. https://vuejs.org/guide/introduction.html, 2026-04-14.

[19] Vite Team. Vite Guide[EB/OL]. https://vite.dev/guide/, 2026-04-14.

[20] Jones M, Bradley J, Sakimura N. JSON Web Token (JWT): RFC 7519[EB/OL]. https://www.rfc-editor.org/rfc/rfc7519, 2026-04-14.